

Programar con IA: flujos de trabajo

Para quien ya programa (o dio sus primeros pasos con IA) y quiere integrar Claude Code, Cursor y GitHub Copilot a su flujo real de trabajo — no solo para prototipos, sino para proyectos serios.

Claude Code

Cursor

GitHub Copilot

Índice

01 Introducción — de construir prototipos a programar en serio con IA

02 Las tres herramientas: Claude Code, Cursor y GitHub Copilot

03 Puesta en marcha: instalación y primer proyecto

04 Prompting técnico: dar el contexto correcto

05 Casos de uso reales: debugging, refactor, tests y documentación

06 Integrarlo a tu flujo diario: git, revisiones y automatización

07 Buenas prácticas y límites

08 Checklist y hacia dónde seguir

09 Recursos — links directos a las herramientas de esta guía

01 · Introducción

Una cosa es pedirle a una IA que construya un prototipo desde cero, y otra muy distinta es usarla todos los días dentro de un proyecto real, con código existente, otras personas trabajando en paralelo, y un historial de decisiones que hay que respetar.

Este es el **Nivel 2** de la serie de Programación/IA de UpgradeLab. Si el Nivel 1 mostró cómo crear algo de la nada sin saber programar, este nivel está pensado para quien ya escribe código (o está aprendiendo a hacerlo) y quiere multiplicar su productividad usando IA como compañera de trabajo diaria, no como reemplazo.

La diferencia central con el Nivel 1 es el **contexto**: en vez de partir de una pantalla en blanco, estas herramientas trabajan sobre proyectos existentes — leen tu código, entienden la estructura de carpetas, respetan convenciones ya establecidas, y hacen cambios quirúrgicos en vez de generar todo de nuevo.

QUÉ VAS A PODER HACER AL TERMINAR ESTA GUÍA

Elegir la herramienta correcta según el tipo de tarea, dar contexto suficiente para que la IA entienda tu proyecto real, usarla para debugging, refactors, tests y documentación, e integrarla a tu flujo diario de git y revisión de código sin perder control sobre lo que se cambia.

02 · Las tres herramientas

No compiten exactamente en lo mismo. Cada una tiene un punto fuerte distinto.

Herramienta	Cómo funciona	Punto fuerte
GitHub Copilot	Autocompletado inteligente dentro de tu editor (VS Code, JetBrains, etc.), sugiere código línea por línea o bloque por bloque mientras escribís	Es el más adoptado y funciona en casi cualquier editor. Ideal para acelerar la escritura del día a día
Cursor	Editor de código propio (basado en VS Code) con IA integrada en todo el flujo: chat, edición multi-archivo, entendimiento profundo del proyecto	Muy fuerte para trabajar sobre proyectos grandes y complejos, con cambios que tocan varios archivos a la vez
Claude Code	Herramienta de terminal: le describís una tarea y trabaja de forma autónoma — lee archivos, escribe código, corre tests, corrige errores por su cuenta	Excelente para tareas grandes y bien delimitadas: refactors extensos, generar tests, automatizar tareas repetitivas

NO ES NECESARIO ELEGIR UNA SOLA

Muchos equipos combinan Copilot para el autocompletado del día a día, con Cursor o Claude Code para tareas más grandes que requieren planificación. Elegí según la tarea, no según la moda.

03 · Puesta en marcha

Los tres tienen una curva de instalación corta. Esto es lo esencial de cada uno.

GitHub Copilot

Se instala como extensión en tu editor (VS Code, JetBrains, Neovim, entre otros) y se activa con una cuenta de GitHub. Requiere suscripción paga luego del período de prueba.

Cursor

Se descarga como aplicación independiente — reemplaza a tu editor actual (aunque podés importar toda tu configuración de VS Code en un paso). Tiene plan gratuito limitado y plan pago con más uso.

Claude Code

Se instala desde la terminal y se usa sin salir de ella. No reemplaza tu editor: trabaja en paralelo, directamente sobre los archivos del proyecto en el que estás parado.

```
PRIMER COMANDO TÍPICO CON CLAUDE CODE, DESDE LA CARPETA DE TU PROYECTO
```

```
Explicame la estructura de este proyecto y qué hace cada carpeta principal
```

Este tipo de comando inicial — pedirle que primero "lea" y explique el proyecto — es una buena práctica antes de pedirle cualquier cambio: te confirma que entendió correctamente el contexto.

04 · Prompting técnico

A diferencia del Nivel 1, acá el contexto técnico específico es lo que determina la calidad del resultado.

Un buen pedido técnico incluye:

1. **Qué archivo(s) o parte del sistema** está involucrado.
2. **Qué comportamiento actual** hay, y qué comportamiento se espera.
3. **Restricciones** — qué no se puede romper, qué convenciones hay que respetar.
4. **Cómo verificar** que el cambio funcionó (un test, un caso de uso concreto).

PROMPT TÉCNICO — FLOJO

Arreglá el bug del login

PROMPT TÉCNICO — BUENO

En `src/lib/actions/auth.ts`, la función `signIn()` esta devolviendo error "Invalid credentials" incluso con contraseña correcta cuando el email tiene mayusculas. Sospecho que es un problema de case-sensitivity contra Supabase. Revisa la funcion, corregi el problema, y no cambies el resto del archivo. Confirmame con que test lo probarias.

Notá que el segundo prompt le da a la IA una hipótesis (no solo el síntoma), delimita el archivo, y le pide una forma de verificar el arreglo — eso reduce enormemente la probabilidad de que "arregle" algo distinto a lo que realmente estaba roto.

Regla practica: cuanto mas grande y ambiguo el pedido, mas probable que la IA tome decisiones de diseño por vos sin que te des cuenta. Para cambios sensibles, pedile primero que te explique su plan antes de ejecutarlo.

05 · Casos de uso reales

Donde estas herramientas realmente ahorran horas, con ejemplos concretos de cómo pedirlo.

Debugging

EJEMPLO

Este test esta fallando: [pegar el error completo]. Encuentra la causa raiz, no soluciones parche, y explicame por que pasaba.

Refactor

EJEMPLO

Esta funcion de 200 lineas en `src/utils/report.ts` hace demasiadas cosas. Dividila en funciones mas chicas con nombres claros, sin cambiar el comportamiento externo. Mostrame el plan antes de aplicarlo.

Generación de tests

EJEMPLO

Genera tests unitarios para `src/lib/pricing.ts`. Cubrí casos limite: precio cero, descuento mayor al 100%, moneda invalida.

Documentación

EJEMPLO

Escribi un README para este proyecto basandote en el codigo actual: que hace, como se instala, como se corre en local, y las variables de entorno que necesita.

En los cuatro casos, el patrón se repite: contexto específico, restricción clara, y una forma de confirmar que el resultado es correcto.

06 · Integrarlo a tu flujo diario

Usar IA de forma aislada, "cuando me acuerdo", rinde mucho menos que integrarla al flujo que ya tenés.

1 **Commits chicos y frecuentes**

Con IA generando código más rápido, es aún más importante commitear en pasos pequeños — así podés revertir fácil si algo no sale como esperabas.

2 **Revisá siempre el diff antes de aceptar**

Ninguna de estas herramientas reemplaza tu criterio. Mirá qué cambió realmente antes de confirmar, sobre todo en archivos que no mencionaste explícitamente.

3 **Usala para las partes tediosas, no para las decisiones importantes**

Generar tests, escribir documentación, o hacer un refactor mecánico son tareas ideales. Decisiones de arquitectura conviene pensarlas vos, con la IA como caja de resonancia.

4 **Automatizá lo repetitivo**

Tareas que hacés siempre igual (generar un componente nuevo con la estructura del proyecto, actualizar un changelog, correr una migración) son buenas candidatas para dejarle un procedimiento fijo a la IA y que lo repita.

07 · Buenas prácticas y límites

- **No le pegues secretos ni claves reales.** Nunca incluyas contraseñas, tokens o claves de API reales en un prompt, aunque sea a una herramienta de confianza.
- **Revisá el código generado antes de deployar a producción,** igual que revisarías el código de cualquier compañero de equipo.
- **No asumás que "compila" significa "es correcto".** Código que corre sin errores puede tener bugs lógicos igual.
- **Cuidado con la sobre-dependencia.** Si dejás de entender el código que tu proyecto tiene porque todo lo generó la IA, se vuelve difícil de mantener a largo plazo. Usala para acelerar, no para tercerizar tu criterio.

08 · Checklist y hacia dónde seguir

- ✓ Sabés cuándo usar Copilot, Cursor o Claude Code según el tipo de tarea.
- ✓ Instalaste al menos una de las tres y probaste un primer comando sobre un proyecto real.
- ✓ Escribís prompts técnicos con contexto, restricciones y forma de verificar el resultado.
- ✓ Usaste IA para debugging, refactor, tests o documentación en un caso real.
- ✓ Integraste la IA a tu flujo de git: commits chicos, revisión de diffs antes de aceptar.

SIGUIENTE PASO

El **Nivel 3 — Proyectos completos** toma todo lo aprendido en los Niveles 1 y 2 y lo aplica a construir un proyecto real de punta a punta: desde la arquitectura inicial hasta el deploy en producción, usando IA en cada etapa del proceso.

09 · Recursos

Links directos a las herramientas mencionadas en esta guía.

- **GitHub Copilot** — github.com/features/copilot
- **Cursor** — cursor.com
- **Claude Code** — claude.com/product/claude-code